

Newton's method,
Limitations of Newton's method,
Damped Newton method,
Backtracking line search,
Quasi-Newton methods

Lecture 5

2026

Newton's Method

Recall that the method of steepest descent uses only first derivatives (gradients) in selecting a suitable search direction. This strategy is not always the most effective. If higher derivatives are used, the resulting iterative algorithm may perform better than the steepest descent method.

Newton's method (sometimes called the Newton-Raphson method) uses first and second derivatives and indeed does perform better than the steepest descent method if the initial point is close to the minimizer.

idea behind this method is as follows. Given a starting point, we construct a quadratic approximation to the objective function that matches the first and second derivative values at that point.

We then minimize the approximate (quadratic) function instead of the original objective function. We use the minimizer of the approximate function as the starting point in the next step and repeat the procedure iteratively.

Newton's Method

Why move beyond Steepest Descent?

- ▶ **Steepest Descent:** Uses only 1st derivatives (can be slow).
- ▶ **Newton's Method:** Uses 1st and 2nd derivatives (Hessian).

The Core Strategy

Approximate the objective function with a **quadratic model** at the current point:

- ▶ Match the gradient and curvature (Hessian).
- ▶ Minimize the approximation to find the next iterate.

Result: Much faster convergence, especially near a minimizer.

Quadratic Approximation: The Core Idea

- ▶ **Quadratic Case:** Newton's method finds the true minimizer in **exactly one step**.
- ▶ **General Case:** The model provides a **local approximation** near the current point.
- ▶ **Iteration:** The predicted minimizer is an estimate; the process is repeated.

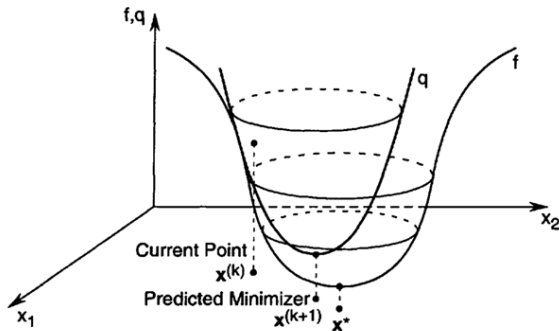


Figure: Local quadratic model matching the original function's curvature.

Derivation of Newton's Method

Let $f : \mathbb{R}^n \rightarrow \mathbb{R}$ be a twice continuously differentiable objective function. Using the second-order Taylor expansion of f about the current iterate $x^{(k)}$, and neglecting terms of order three and higher, we obtain the quadratic model

$$f(x) \approx f(x^{(k)}) + \nabla f(x^{(k)})^T (x - x^{(k)}) + \frac{1}{2} (x - x^{(k)})^T \nabla^2 f(x^{(k)}) (x - x^{(k)}) =: q(x)$$

For simplicity, let

$$g^{(k)} = \nabla f(x^{(k)}).$$

Applying the first-order necessary condition to $q(x)$, we get

$$\nabla q(x) = g^{(k)} + \nabla^2 f(x^{(k)}) (x - x^{(k)}) = 0.$$

If $\nabla^2 f(x^{(k)})$ is positive definite, then q has a minimizer at

$$x^{(k+1)} = x^{(k)} - (\nabla^2 f(x^{(k)}))^{-1} g^{(k)}.$$

This recursive formula is called **Newton's method**.

Newton's Method Algorithm

Step 1. Choose an initial point $x^{(0)}$ and set $k = 0$.

Step 2. Compute the gradient $\nabla f(x^{(k)})$ and the Hessian $\nabla^2 f(x^{(k)})$.

Step 3. If $\nabla f(x^{(k)}) = 0$, or if a selected stopping criterion is satisfied, stop.

Step 4. Update

$$x^{(k+1)} = x^{(k)} - (\nabla^2 f(x^{(k)}))^{-1} \nabla f(x^{(k)}).$$

Set $k \leftarrow k + 1$ and return to Step 2.

Newton Direction

The iteration formula

$$x^{(k+1)} = x^{(k)} - (\nabla^2 f(x^{(k)}))^{-1} \nabla f(x^{(k)})$$

can be written in a more practical form.

Define the **Newton direction** $p^{(k)}$ as the solution of the linear system

$$\nabla^2 f(x^{(k)}) p^{(k)} = -\nabla f(x^{(k)}).$$

Then the Newton iterate can be written as

$$x^{(k+1)} = x^{(k)} + p^{(k)}.$$

- ▶ Thus, Newton's method requires solving a linear system at each iteration.
- ▶ In practice, we do not compute the inverse Hessian explicitly.
- ▶ Instead, we solve the system for $p^{(k)}$.

Newton's Method Algorithm Using Newton Direction

Step 1. Choose an initial point $x^{(0)}$ and set $k = 0$.

Step 2. Compute the gradient $\nabla f(x^{(k)})$ and the Hessian $\nabla^2 f(x^{(k)})$.

Step 3. If $\nabla f(x^{(k)}) = 0$, or if a selected stopping criterion is satisfied, stop.

Step 4. Compute the Newton direction $p^{(k)}$ by solving

$$\nabla^2 f(x^{(k)}) p^{(k)} = -\nabla f(x^{(k)}).$$

Step 5. Update

$$x^{(k+1)} = x^{(k)} + p^{(k)}.$$

Set $k \leftarrow k + 1$ and return to Step 2.

Newton's Method in One Dimension

Consider the unconstrained minimization problem

$$\min_{x \in \mathbb{R}} f(x),$$

where $f : \mathbb{R} \rightarrow \mathbb{R}$ is twice continuously differentiable. Using the second-order Taylor expansion of f about the current iterate x_k , we obtain

$$f(x) \approx f(x_k) + f'(x_k)(x - x_k) + \frac{1}{2}f''(x_k)(x - x_k)^2.$$

Setting the derivative of this quadratic model equal to zero gives

$$f'(x_k) + f''(x_k)(x - x_k) = 0.$$

Hence,

$$x_{k+1} = x_k - \frac{f'(x_k)}{f''(x_k)}.$$

Example: Newton's Method in One Dimension

Consider the minimization problem

$$\min_{x \in \mathbb{R}} f(x), \quad f(x) = x^2 + 2x + 2.$$

Compute the first and second derivatives:

$$f'(x) = 2x + 2, \quad f''(x) = 2.$$

The Newton iteration is

$$x_{k+1} = x_k - \frac{f'(x_k)}{f''(x_k)}.$$

Substituting the derivatives gives

$$x_{k+1} = x_k - \frac{2x_k + 2}{2} = x_k - (x_k + 1) = -1.$$

Thus, Newton's method finds the minimizer $x^* = -1$ in one step, regardless of the starting point.

Geometric Interpretation of Newton's Method

At the current iterate x_k , we approximate the function $f(x)$ by the second-order Taylor polynomial

$$f(x) \approx f(x_k) + f'(x_k)(x - x_k) + \frac{1}{2}f''(x_k)(x - x_k)^2.$$

The minimizer of this quadratic model gives the next iterate

$$x_{k+1} = x_k - \frac{f'(x_k)}{f''(x_k)}.$$

- ▶ The step depends on both the first and second derivatives.
- ▶ If $f''(x_k) > 0$, the step moves toward a local minimizer.
- ▶ The function f is locally approximated by a quadratic curve.
- ▶ The minimum of this quadratic approximation determines the next point.
- ▶ Repeating this procedure generates the Newton sequence.

Quadratic Convergence of Newton's Method

One of the main advantages of Newton's method is its very fast local convergence. Suppose that x^* is a strict local minimizer of f , that

$$\nabla f(x^*) = 0, \quad \nabla^2 f(x^*) \text{ is positive definite,}$$

Then, if $x^{(0)}$ is sufficiently close to x^* , Newton's method converges quadratically.

- ▶ The error at each step is roughly proportional to the square of the previous error.
- ▶ Therefore, once the iterates are close to the solution, the convergence is very rapid.

Newton's Method vs. Steepest Descent

Both methods are iterative techniques for unconstrained optimization, but they use different information about the objective function.

- ▶ **Steepest descent** uses only the gradient $\nabla f(x^{(k)})$.
- ▶ **Newton's method** uses both the gradient $\nabla f(x^{(k)})$ and the Hessian $\nabla^2 f(x^{(k)})$.

Consequently,

- ▶ Steepest descent is simpler and cheaper per iteration,
- ▶ Newton's method is usually much faster near a minimizer.

In general,

Steepest descent: linear convergence,

Newton's method: quadratic convergence (locally).

Thus, Newton's method often requires fewer iterations, although each iteration is more expensive.

Example: Newton's Method in Two Variables

Consider the minimization problem

$$\min_{(x,y) \in \mathbb{R}^2} f(x,y), \quad f(x,y) = x^2 + y^2 + xy - 2x - 3y.$$

Compute the gradient

$$\nabla f(x,y) = \begin{bmatrix} 2x + y - 2 \\ 2y + x - 3 \end{bmatrix}$$

and the Hessian

$$\nabla^2 f(x,y) = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}.$$

Newton's iteration is

$$x^{(k+1)} = x^{(k)} - (\nabla^2 f(x^{(k)}))^{-1} \nabla f(x^{(k)}).$$

Since f is quadratic, Newton's method finds the minimizer in one iteration.

Example (continued): Using the Inverse Hessian

For the initial point $x^{(0)} = [0 \ 0]^T$, we have

$$\nabla f(x^{(0)}) = \begin{bmatrix} -2 \\ -3 \end{bmatrix}, \quad \nabla^2 f(x^{(0)}) = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}.$$

Using Newton's iteration formula

$$x^{(1)} = x^{(0)} - (\nabla^2 f(x^{(0)}))^{-1} \nabla f(x^{(0)}),$$

we first compute

$$(\nabla^2 f(x^{(0)}))^{-1} = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}^{-1} = \frac{1}{3} \begin{bmatrix} 2 & -1 \\ -1 & 2 \end{bmatrix}.$$

Hence,

$$x^{(1)} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} - \frac{1}{3} \begin{bmatrix} 2 & -1 \\ -1 & 2 \end{bmatrix} \begin{bmatrix} -2 \\ -3 \end{bmatrix} = \begin{bmatrix} \frac{1}{3} \\ \frac{4}{3} \end{bmatrix}.$$

Therefore, the next iterate is

$$x^{(1)} = \begin{bmatrix} \frac{1}{3} \\ \frac{4}{3} \end{bmatrix}.$$

Example (continued): Using the Newton Direction

The same iteration can also be computed without forming the inverse Hessian explicitly.

From the previous slide, $\nabla f(x^{(0)}) = [-2 \ -3]^T$, $\nabla^2 f(x^{(0)}) = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$.

The Newton direction satisfies

$$\nabla^2 f(x^{(0)}) p^{(0)} = -\nabla f(x^{(0)}),$$

so we solve

$$\begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix} p^{(0)} = \begin{bmatrix} 2 \\ 3 \end{bmatrix}$$

and obtain

$$p^{(0)} = \begin{bmatrix} \frac{1}{3} \\ \frac{4}{3} \end{bmatrix}.$$

Hence

$$x^{(1)} = x^{(0)} + p^{(0)} = \begin{bmatrix} 1 \\ \frac{4}{3} \end{bmatrix}.$$

This is the same iterate as before, but obtained without explicitly computing the inverse Hessian.

A Limitation of Newton's Method

Although Newton's method converges very rapidly near a minimizer, it also has some practical limitations.

- ▶ Newton's method requires computing the Hessian matrix $\nabla^2 f(x^{(k)})$.
- ▶ This can be expensive for high-dimensional problems.
- ▶ The Hessian may not always be positive definite.

If $\nabla^2 f(x^{(k)})$ is not positive definite, the Newton direction may fail to be a descent direction.

As a result, the method may move away from the minimizer or even diverge.

To overcome this difficulty, modified versions of Newton's method are often used in practice.

Damped Newton Method

To improve the global behavior of Newton's method, a step-size parameter is often introduced. Instead of using the full Newton step

$$x^{(k+1)} = x^{(k)} + p^{(k)},$$

we use

$$x^{(k+1)} = x^{(k)} + \alpha_k p^{(k)}, \quad 0 < \alpha_k \leq 1,$$

where $p^{(k)}$ is the Newton direction defined by

$$\nabla^2 f(x^{(k)}) p^{(k)} = -\nabla f(x^{(k)}).$$

The parameter α_k is chosen to ensure sufficient decrease in the objective function.

- ▶ If $\alpha_k = 1$, we recover the classical Newton method.
- ▶ If $0 < \alpha_k < 1$, the step is shortened.
- ▶ This often improves stability and prevents divergence.

Backtracking Line Search

Step 1. Choose parameters $0 < \rho < 1$, $0 < c < 1$, and an initial step size $\alpha = 1$, where ρ is the reduction factor controlling how aggressively the step size is shortened (typically $\rho = \frac{1}{2}$), and c is the Armijo parameter controlling the amount of sufficient decrease required for acceptance (typically $c = 10^{-4}$).

Step 2. Compute a descent direction $p^{(k)}$.

Step 3. If

$$f(x^{(k)} + \alpha p^{(k)}) \leq f(x^{(k)}) + c \alpha \nabla f(x^{(k)})^T p^{(k)},$$

accept the step size $\alpha_k = \alpha$.

Step 4. Otherwise, reduce the step size

$$\alpha \leftarrow \rho \alpha$$

and return to Step 3.

Note that c appears in the Armijo condition, whereas ρ is only used to shrink the trial step size when the condition is not satisfied.

Armijo Condition (Sufficient Decrease)

A step size $\alpha > 0$ is accepted if it satisfies

$$f(x^{(k)} + \alpha p^{(k)}) \leq f(x^{(k)}) + c \alpha \nabla f(x^{(k)})^T p^{(k)}, \quad 0 < c < 1.$$

This condition ensures that the objective function decreases sufficiently compared to the linear prediction.

In practice:

- ▶ Start with $\alpha = 1$,
- ▶ Reduce $\alpha \leftarrow \rho \alpha$ if the condition fails,
- ▶ Repeat until it is satisfied.

Example: Classical Newton Step May Fail

Consider the minimization problem

$$\min_{(x,y) \in \mathbb{R}^2} f(x,y), \quad f(x,y) = e^x - 3x + \frac{1}{2}y^2.$$

The gradient and Hessian are

$$\nabla f(x,y) = \begin{bmatrix} e^x - 3 \\ y \end{bmatrix}, \quad \nabla^2 f(x,y) = \begin{bmatrix} e^x & 0 \\ 0 & 1 \end{bmatrix}.$$

Choose the initial point $x^{(0)} = \begin{bmatrix} -1 \\ 1 \end{bmatrix}$. Then,

$$\nabla f(x^{(0)}) = \begin{bmatrix} e^{-1} - 3 \\ 1 \end{bmatrix} \approx \begin{bmatrix} -2.6321 \\ 1 \end{bmatrix},$$

$$\nabla^2 f(x^{(0)}) = \begin{bmatrix} e^{-1} & 0 \\ 0 & 1 \end{bmatrix} \approx \begin{bmatrix} 0.3679 & 0 \\ 0 & 1 \end{bmatrix}.$$

Example (continued): Iteration 0: Classical Newton Step

The Newton direction $p^{(0)}$ satisfies

$$\nabla^2 f(x^{(0)}) p^{(0)} = -\nabla f(x^{(0)}).$$

Since the Hessian is diagonal, $p^{(k)} = \begin{bmatrix} 3e^{-x_k} - 1 \\ -y_k \end{bmatrix}$.

$$p^{(0)} = - \begin{bmatrix} e^{-1} & 0 \\ 0 & 1 \end{bmatrix}^{-1} \begin{bmatrix} e^{-1} - 3 \\ 1 \end{bmatrix} = \begin{bmatrix} 3e - 1 \\ -1 \end{bmatrix} \approx \begin{bmatrix} 7.1548 \\ -1 \end{bmatrix}.$$

Using the full Newton step,

$$x^{(1)} = x^{(0)} + p^{(0)} = \begin{bmatrix} -1 \\ 1 \end{bmatrix} + \begin{bmatrix} 7.1548 \\ -1 \end{bmatrix} = \begin{bmatrix} 6.1548 \\ 0 \end{bmatrix}.$$

Now,

$$f(x^{(0)}) = e^{-1} + 3 + \frac{1}{2} \approx 3.8679,$$

while

$$f(x^{(1)}) = e^{6.1548} - 3(6.1548) \approx 452.5295.$$

So the full Newton step causes a dramatic increase in the objective function.

Example (continued): Why Newton Fails Here?

In the previous slide, we saw that $f(x^{(1)}) \approx 452.5$, while $f(x^{(0)}) \approx 3.8$.

The Reason for Failure:

- ▶ **Model Mismatch:** Newton's method "assumes" the objective function is locally **quadratic**.
- ▶ **Growth Rate:** However, e^x grows much faster than any quadratic function.
- ▶ **Overshooting:** Because the quadratic model underestimates the actual steepness of e^x further away, the predicted step "jumps" too far into a region where the function value is much higher.

Key Insight

Newton's method lacks a "global perspective." It trusts its local quadratic approximation implicitly, which can lead to massive overshooting in functions with high-order growth like exponentials.

Step-by-Step Backtracking: Initial Data

To check the Armijo condition:

$$f(x^{(k)} + \alpha p^{(k)}) \leq f(x^{(0)}) + c\alpha \nabla f(x^{(0)})^T p^{(0)}$$

Fixed Values for Iteration 0:

- ▶ Initial function value: $f(x^{(0)}) \approx 3.8679$
- ▶ Descent direction dot product:

$$\nabla f(x^{(0)})^T p^{(0)} \approx \begin{bmatrix} -2.6321 & 1 \end{bmatrix} \begin{bmatrix} 7.1548 \\ -1 \end{bmatrix} \approx -19.8310$$

- ▶ Parameters: $c = 10^{-4}$ and $\rho = 0.5$ (reduction factor).

The Acceptance Threshold (RHS)

$$RHS(\alpha) = 3.8679 + \alpha \cdot (10^{-4} \cdot -19.8310) = 3.8679 - \alpha \cdot 0.00198$$

Trial 1: Full Newton Step ($\alpha = 1$)

1. Compute New Point:

$$x^{(0)} + 1 \cdot p^{(0)} = \begin{bmatrix} -1 \\ 1 \end{bmatrix} + \begin{bmatrix} 7.1548 \\ -1 \end{bmatrix} = \begin{bmatrix} 6.1548 \\ 0 \end{bmatrix}$$

2. Check Armijo Condition:

- ▶ **LHS:** $f(6.1548, 0) = e^{6.1548} - 3(6.1548) + 0 \approx \mathbf{452.5295}$
- ▶ **RHS:** $3.8679 - (1 \cdot 0.00198) \approx \mathbf{3.8659}$

Decision: REJECT

$452.5295 \not\leq 3.8659$. The function value exploded due to overshooting the exponential growth. We reduce $\alpha \leftarrow \alpha \cdot 0.5$.

Trial 2: First Backtrack ($\alpha = 0.5$)

1. Compute New Point:

$$x^{(0)} + 0.5 \cdot p^{(0)} = \begin{bmatrix} -1 \\ 1 \end{bmatrix} + \begin{bmatrix} 3.5774 \\ -0.5 \end{bmatrix} = \begin{bmatrix} 2.5774 \\ 0.5 \end{bmatrix}$$

2. Check Armijo Condition:

► LHS:

$$f(2.5774, 0.5) = e^{2.5774} - 3(2.5774) + \frac{1}{2}(0.5)^2 \approx \mathbf{5.5559}$$

► RHS: $3.8679 - (0.5 \cdot 0.00198) \approx \mathbf{3.8669}$

Decision: REJECT

$5.5559 \not\leq 3.8669$. Although the value decreased significantly from 452, it is still higher than our starting point. We reduce α again.

Trial 3: Second Backtrack ($\alpha = 0.25$)

1. Compute New Point:

$$x^{(0)} + 0.25 \cdot p^{(0)} = \begin{bmatrix} -1 \\ 1 \end{bmatrix} + \begin{bmatrix} 1.7887 \\ -0.25 \end{bmatrix} = \begin{bmatrix} 0.7887 \\ 0.75 \end{bmatrix}$$

2. Check Armijo Condition:

► LHS:

$$f(0.7887, 0.75) = e^{0.7887} - 3(0.7887) + \frac{1}{2}(0.75)^2 \approx \mathbf{0.1157}$$

$$\text{► RHS: } 3.8679 - (0.25 \cdot 0.00198) \approx \mathbf{3.8674}$$

Decision: **ACCEPT**

$0.1157 \leq 3.8674$. The condition is satisfied.

$$x^{(1)} = \begin{bmatrix} 0.7887 \\ 0.75 \end{bmatrix}$$

Summary of Iteration 0

We test the step sizes $\alpha = 1, \frac{1}{2}, \frac{1}{4}, \dots$ until the Armijo condition is satisfied.

For $\alpha = 1$,

$$x^{(0)} + \alpha p^{(0)} = \begin{bmatrix} 6.1548 \\ 0 \end{bmatrix}, \quad f(x^{(0)} + p^{(0)}) \approx 452.5295.$$

This is rejected.

For $\alpha = \frac{1}{2}$,

$$x^{(0)} + \frac{1}{2}p^{(0)} = \begin{bmatrix} 2.5774 \\ 0.5 \end{bmatrix}, \quad f\left(x^{(0)} + \frac{1}{2}p^{(0)}\right) \approx 5.5559.$$

This is also rejected.

For $\alpha = \frac{1}{4}$,

$$x^{(0)} + \frac{1}{4}p^{(0)} = \begin{bmatrix} 0.7887 \\ 0.75 \end{bmatrix}, \quad f\left(x^{(0)} + \frac{1}{4}p^{(0)}\right) \approx 0.1157.$$

This step is accepted. Therefore, $x^{(1)} = \begin{bmatrix} 0.7887 \\ 0.75 \end{bmatrix}$.

Iteration 1:

Now take $x^{(1)} = \begin{bmatrix} 0.7887 \\ 0.75 \end{bmatrix}$. Then:

$$\nabla f(x^{(1)}) = \begin{bmatrix} e^{0.7887} - 3 \\ 0.75 \end{bmatrix} \approx \begin{bmatrix} -0.7995 \\ 0.75 \end{bmatrix}, \quad \nabla^2 f(x^{(1)}) = \begin{bmatrix} e^{0.7887} & 0 \\ 0 & 1 \end{bmatrix} \approx \begin{bmatrix} 2.2005 & 0 \\ 0 & 1 \end{bmatrix}.$$

Hence

$$p^{(1)} = - \left(\nabla^2 f(x^{(1)}) \right)^{-1} \nabla f(x^{(1)}) \approx \begin{bmatrix} 0.3633 \\ -0.75 \end{bmatrix}.$$

Trying $\alpha = 1$,

$$x^{(2)} = x^{(1)} + p^{(1)} = \begin{bmatrix} 1.1520 \\ 0 \end{bmatrix}, \quad f(x^{(2)}) \approx -0.2915.$$

Check Armijo Condition ($\alpha = 1$):

► **LHS:** $f(x^{(2)}) \approx -0.2915$

► **RHS:** $f(x^{(1)}) + c\alpha \nabla f(x^{(1)})^T p^{(1)}$

$$\approx 0.1157 + 10^{-4} \cdot 1 \cdot ((-0.7995)(0.3633) + (0.75)(-0.75))$$

$$\approx 0.1157 - 0.000085 \approx \mathbf{0.1156}$$

Since $-0.2915 \leq 0.1156$, the condition $\alpha_1 = 1$ is accepted.

Iteration 2

At

$$x^{(2)} = \begin{bmatrix} 1.1520 \\ 0 \end{bmatrix},$$

we compute

$$\nabla f(x^{(2)}) = \begin{bmatrix} e^{1.1520} - 3 \\ 0 \end{bmatrix} \approx \begin{bmatrix} 0.1567 \\ 0 \end{bmatrix}.$$

The Newton direction becomes

$$p^{(2)} = \begin{bmatrix} 3e^{-1.1520} - 1 \\ 0 \end{bmatrix} \approx \begin{bmatrix} -0.0520 \\ 0 \end{bmatrix}.$$

Using $\alpha = 1$,

$$x^{(3)} = x^{(2)} + p^{(2)} = \begin{bmatrix} 1.1000 \\ 0 \end{bmatrix},$$

and

$$f(x^{(3)}) \approx -0.2958.$$

$\alpha_2 = 1$ satisfies the Armijo condition; thus, it is accepted as the step size.

Iteration 3

At

$$x^{(3)} = \begin{bmatrix} 1.1000 \\ 0 \end{bmatrix},$$

we obtain

$$\nabla f(x^{(3)}) = \begin{bmatrix} e^{1.1000} - 3 \\ 0 \end{bmatrix} \approx \begin{bmatrix} 0.0042 \\ 0 \end{bmatrix},$$

and

$$p^{(3)} \approx \begin{bmatrix} -0.0014 \\ 0 \end{bmatrix}.$$

Again $\alpha = 1$ is accepted, giving

$$x^{(4)} = \begin{bmatrix} 1.0986 \\ 0 \end{bmatrix}, \quad f(x^{(4)}) \approx -0.2958369.$$

This is already extremely close to the minimizer.

Iteration 4: Convergence to the Minimizer

One more Newton step gives

$$x^{(5)} \approx \begin{bmatrix} 1.0986123 \\ 0 \end{bmatrix},$$

which coincides numerically with the exact minimizer

$$x^* = \begin{bmatrix} \ln 3 \\ 0 \end{bmatrix} \approx \begin{bmatrix} 1.0986123 \\ 0 \end{bmatrix}.$$

Indeed, the first-order optimality condition is

$$\nabla f(x, y) = \begin{bmatrix} e^x - 3 \\ y \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix},$$

so

$$e^x = 3, \quad y = 0, \quad x = \ln 3.$$

Therefore the damped Newton method converges to the minimizer.

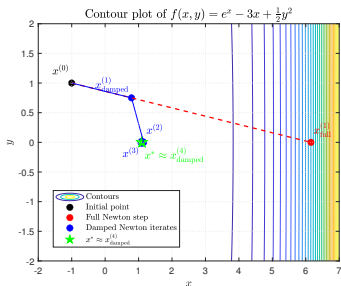
Summary of the Damped Newton Iterations

k	$x^{(k)}$	$p^{(k)}$	α_k	$f(x^{(k)})$
0	$\begin{bmatrix} -1 \\ 1 \end{bmatrix}$	$\begin{bmatrix} 7.1548 \\ -1 \end{bmatrix}$	$\frac{1}{4}$	3.8679
1	$\begin{bmatrix} 0.7887 \\ 0.75 \end{bmatrix}$	$\begin{bmatrix} 0.3633 \\ -0.75 \end{bmatrix}$	1	0.1157
2	$\begin{bmatrix} 1.1520 \\ 0 \end{bmatrix}$	$\begin{bmatrix} -0.0520 \\ 0 \end{bmatrix}$	1	-0.2915
3	$\begin{bmatrix} 1.1000 \\ 0 \end{bmatrix}$	$\begin{bmatrix} -0.0014 \\ 0 \end{bmatrix}$	1	-0.2958
4	$\begin{bmatrix} 1.0986 \\ 0 \end{bmatrix}$	$\approx \begin{bmatrix} 0 \\ 0 \end{bmatrix}$	1	-0.2958369

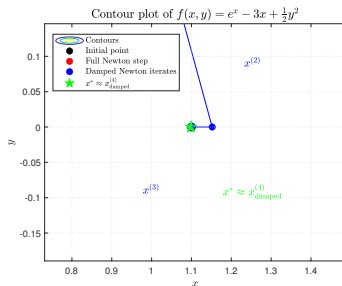
The iterates converge to $x^* = \begin{bmatrix} \ln 3 \\ 0 \end{bmatrix}$.

Geometric Interpretation of the Newton and Damped Newton Steps

The contour plots illustrate that the full Newton step moves far away from the minimizer, whereas the damped Newton iterates provide safer updates toward the minimum.



Wide view



Zoomed view near the minimizer

The figures show the initial point $x^{(0)} = (-1, 1)^T$, the full Newton iterate $x_{\text{full}}^{(1)} = (6.1548, 0)^T$, the damped Newton iterate $x_{\text{damped}}^{(1)} = (0.7887, 0.75)^T$, and the minimizer $x^* = (\ln 3, 0)^T$.

Damped Newton Method

Step 1. Choose an initial point $x^{(0)}$ and set $k = 0$.

Step 2. Compute the gradient $\nabla f(x^{(k)})$ and the Hessian $\nabla^2 f(x^{(k)})$.

Step 3. If $\nabla f(x^{(k)}) = 0$, or if a selected stopping criterion is satisfied, stop.

Step 4. Compute the Newton direction $p^{(k)}$ by solving

$$\nabla^2 f(x^{(k)}) p^{(k)} = -\nabla f(x^{(k)}).$$

Step 5. Choose a step size α_k using a line search procedure.

Step 6. Update

$$x^{(k+1)} = x^{(k)} + \alpha_k p^{(k)}.$$

Set $k \leftarrow k + 1$ and return to Step 2.

Damped Newton Method

Step 1. Choose an initial point $x^{(0)}$ and set $k = 0$.

Step 2. Compute the gradient $\nabla f(x^{(k)})$ and the Hessian $\nabla^2 f(x^{(k)})$.

Step 3. If $\nabla f(x^{(k)}) = 0$, or if a selected stopping criterion is satisfied, stop.

Step 4. Compute the Newton direction $p^{(k)}$ by solving

$$\nabla^2 f(x^{(k)}) p^{(k)} = -\nabla f(x^{(k)}).$$

Step 5. Choose a step size α_k using a line search procedure.

Step 6. Update

$$x^{(k+1)} = x^{(k)} + \alpha_k p^{(k)}.$$

Set $k \leftarrow k + 1$ and return to Step 2.

Summary of Descent and Newton-Type Methods

We have now seen three important iterative methods for unconstrained optimization.

- ▶ **Steepest descent method:** uses the negative gradient direction

$$p^{(k)} = -\nabla f(x^{(k)}).$$

- ▶ **Newton's method:** uses second-order information and computes the direction from

$$\nabla^2 f(x^{(k)}) p^{(k)} = -\nabla f(x^{(k)}).$$

- ▶ **Damped Newton method:** updates the iterate by

$$x^{(k+1)} = x^{(k)} + \alpha_k p^{(k)}, \quad 0 < \alpha_k \leq 1,$$

where α_k is chosen by a line search procedure.

In general, Newton-type methods are more expensive per iteration, but they are often much faster near a minimizer.

Quasi-Newton Methods

Can we preserve the power of Newton's method and achieve Newton-like convergence without explicitly computing the exact Hessian $\nabla^2 f(x^{(k)})$ at each iteration?

- ▶ **Computational Relief:** Avoids n^2 second-derivative evaluations and solving a linear system ($O(n^3)$) at every iteration.
- ▶ **Efficiency:** Uses only **gradient information** from current and previous steps to "learn" the curvature.
- ▶ **Stability:** Under suitable update rules and line search conditions, the approximation can be kept positive definite, ensuring a descent direction.

Quasi-Newton methods bridge the gap between Steepest Descent (low cost) and Newton's Method (high speed) and approximate the Hessian matrix, or its inverse, instead of computing it exactly.

Basic Idea of Quasi-Newton Methods

Quasi-Newton methods construct an approximation of the Hessian matrix (or its inverse) using information from previous iterations.

Instead of computing the true Hessian $\nabla^2 f(x^{(k)})$, we update an approximation $B^{(k)} \approx \nabla^2 f(x^{(k)})$ (or its inverse $H^{(k)}$) at each iteration.

The search direction is then determined by

$$B^{(k)} p^{(k)} = -\nabla f(x^{(k)}) \quad \text{or equivalently} \quad p^{(k)} = -H^{(k)} \nabla f(x^{(k)}).$$

The Secant Condition: After moving from $x^{(k)}$ to $x^{(k+1)}$, the new approximation $B^{(k+1)}$ must satisfy:

$$B^{(k+1)} s^{(k)} = y^{(k)}$$

where:

- ▶ $s^{(k)} = x^{(k+1)} - x^{(k)}$ (change in position)
- ▶ $y^{(k)} = \nabla f(x^{(k+1)}) - \nabla f(x^{(k)})$ (change in gradient)

This ensures that $B^{(k+1)}$ behaves like the true Hessian in the most recently taken step direction, i.e. in the direction of the latest displacement $s^{(k)}$.

Why the Secant Condition?

The secant condition is a multi-dimensional generalization of the 1D secant method.

In 1D, for a function $g(x) = f'(x)$, the derivative (Hessian of f) is approximated by:

$$f''(x_{k+1}) \approx \frac{f'(x_{k+1}) - f'(x_k)}{x_{k+1} - x_k} = \frac{y_k}{s_k}$$

The Logic:

- Taylor expansion of the gradient:

$$\nabla f(x^{(k+1)}) - \nabla f(x^{(k)}) = \nabla^2 f(\xi^{(k)}) (x^{(k+1)} - x^{(k)})$$

for some $\xi^{(k)}$ between $x^{(k)}$ and $x^{(k+1)}$.

This relation is called the **secant condition**. We force $B^{(k+1)}$ to make this approximation **exact** along the direction $s^{(k)}$.

This ensures our model "learns" the true curvature of the function using only the history of gradient changes.

The DFP Update

The **Davidon–Fletcher–Powell (DFP)** method was the first functional quasi-Newton algorithm.

Key Idea: Instead of approximating the Hessian (B), we directly update the **inverse Hessian approximation** ($H \approx [\nabla^2 f]^{-1}$).

- ▶ **Search Direction:** $p^{(k)} = -H^{(k)} \nabla f(x^{(k)})$ (No matrix inversion needed!)
- ▶ **The Update:** We add two rank-1 matrices to $H^{(k)}$ to get $H^{(k+1)}$:

$$H^{(k+1)} = H^{(k)} + \underbrace{\frac{s^{(k)}(s^{(k)})^T}{(s^{(k)})^T y^{(k)}}}_{\text{Corrective Term}} - \underbrace{\frac{H^{(k)} y^{(k)} (y^{(k)})^T H^{(k)}}{(y^{(k)})^T H^{(k)} y^{(k)}}}_{\text{Damping Term}}$$

Why these terms?

- ▶ Guarantees the **inverse secant condition**: $H^{(k+1)} y^{(k)} = s^{(k)}$.
- ▶ Preserves **symmetry** and (under line search) **positive definiteness**.

Numerical Example: Iteration 1 (Initialization)

Minimize $f(x, y) = x^2 + y^2 + xy - 2x - 3y$ starting from $x^{(0)} = [0, 0]^T$.

1. Initial Search Direction: With $H^{(0)} = I$ and $\nabla f(x^{(0)}) = [-2, -3]^T$:

$$p^{(0)} = -I\nabla f(x^{(0)}) = \begin{bmatrix} 2 \\ 3 \end{bmatrix}$$

2. Optimal Step Size (α_0): We find α that minimizes $\phi(\alpha) = f(x^{(0)} + \alpha p^{(0)})$: **Exact line search:** Substitute $x = 2\alpha, y = 3\alpha$ into $f \implies \phi(\alpha) = 19\alpha^2 - 13\alpha$. Set $\phi'(\alpha) = 38\alpha - 13 = 0 \implies \alpha_0 = 13/38$.

3. Update to $x^{(1)}$:

$$x^{(1)} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} + \frac{13}{38} \begin{bmatrix} 2 \\ 3 \end{bmatrix} = \begin{bmatrix} 13/19 \\ 39/38 \end{bmatrix} \approx \begin{bmatrix} 0.68 \\ 1.03 \end{bmatrix}$$

The first step is identical to Steepest Descent because $H^{(0)} = I$.

Numerical Example: Iteration 2 (Information Gain)

We calculate the differences in position (s) and gradient (y) between iterations:

1. Position Change ($s^{(0)}$):

$$s^{(0)} = x^{(1)} - x^{(0)} = \begin{bmatrix} 13/19 \\ 39/38 \end{bmatrix} - \begin{bmatrix} 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 13/19 \\ 39/38 \end{bmatrix} \approx \begin{bmatrix} 0.68 \\ 1.03 \end{bmatrix}$$

2. Gradient Change ($y^{(0)}$): Recall $\nabla f(x, y) = \begin{bmatrix} 2x + y - 2 \\ x + 2y - 3 \end{bmatrix}$.

$$\blacktriangleright \nabla f(x^{(0)}) = \begin{bmatrix} -2 \\ -3 \end{bmatrix}, \quad \nabla f(x^{(1)}) = \begin{bmatrix} 15/38 \\ -5/19 \end{bmatrix} \approx \begin{bmatrix} 0.39 \\ -0.26 \end{bmatrix}$$

$$y^{(0)} = \nabla f(x^{(1)}) - \nabla f(x^{(0)}) = \begin{bmatrix} 15/38 - (-2) \\ -5/19 - (-3) \end{bmatrix} = \begin{bmatrix} 91/38 \\ 52/19 \end{bmatrix} \approx \begin{bmatrix} 2.39 \\ 2.74 \end{bmatrix}$$

3. The Scalar Curvature: $(s^{(0)})^T y^{(0)} = \frac{169}{38} \approx 4.45$

This scalar tells us how much $f(x, y)$ "curved" along the direction $p^{(0)}$.

Numerical Example: Iteration 2 (The DFP Update)

Now we update $H^{(0)} = I$ to $H^{(1)}$ using the two Rank-1 components:

$$H^{(1)} = I + \frac{s^{(0)}(s^{(0)})^T}{(s^{(0)})^T y^{(0)}} - \frac{y^{(0)}(y^{(0)})^T}{(y^{(0)})^T y^{(0)}}.$$

Term 1 (Correction): $\frac{ss^T}{s^T y} \approx \begin{bmatrix} 0.1053 & 0.1579 \\ 0.1579 & 0.2368 \end{bmatrix}$

Term 2 (Damping): $\frac{Hyy^T H}{y^T Hy} \approx \begin{bmatrix} 0.4336 & 0.4956 \\ 0.4956 & 0.5664 \end{bmatrix}$

Final Result:

$$H^{(1)} \approx \begin{bmatrix} 1.0 & 0 \\ 0 & 1.0 \end{bmatrix} + \begin{bmatrix} 0.1053 & 0.1579 \\ 0.1579 & 0.2368 \end{bmatrix} - \begin{bmatrix} 0.4336 & 0.4956 \\ 0.4956 & 0.5664 \end{bmatrix}$$

$$H^{(1)} \approx \begin{bmatrix} 0.6716 & -0.3377 \\ -0.3377 & 0.6705 \end{bmatrix}.$$

Numerical Example: Iteration 2 (Next point)

Now we use the updated inverse Hessian $H^{(1)}$ to find the next point $x^{(2)}$.

1. New Search Direction ($p^{(1)}$): Using $\nabla f(x^{(1)}) \approx \begin{bmatrix} 0.39 \\ -0.26 \end{bmatrix}$ and our updated $H^{(1)}$:

$$p^{(1)} = -H^{(1)}\nabla f(x^{(1)}) \approx - \begin{bmatrix} 0.6716 & -0.3377 \\ -0.3377 & 0.6705 \end{bmatrix} \begin{bmatrix} 0.39 \\ -0.26 \end{bmatrix} \approx \begin{bmatrix} -0.3540 \\ 0.3097 \end{bmatrix}$$

2. Update to $x^{(2)}$: Assuming an exact line search gives $\alpha_1 \approx 0.9912$:

$$x^{(2)} = x^{(1)} + \alpha_1 p^{(1)} = \begin{bmatrix} 13/19 \\ 39/38 \end{bmatrix} + \frac{113}{114} \begin{bmatrix} -40/113 \\ 35/113 \end{bmatrix} = \begin{bmatrix} 1/3 \\ 4/3 \end{bmatrix}.$$

3. Objective Function Progress:

- ▶ $f(x^{(0)}) = f(0,0) = 0$
- ▶ $f(x^{(1)}) = -169/76 \approx -2.2237$
- ▶ $f(x^{(2)}) = -7/3 \approx -2.3333$

Numerical Example: Summary of Progress

The true minimizer of $f(x, y)$ is

$$x^* = \begin{bmatrix} 1/3 \\ 4/3 \end{bmatrix} \approx \begin{bmatrix} 0.33 \\ 1.33 \end{bmatrix}.$$

Iteration	Point (x, y)	$f(x, y)$	Status
$k = 0$	$(0, 0)$	0	Starting Point
$k = 1$	$(0.68, 1.03)$	-2.22	First DFP Step
$k = 2$	$(1/3, 4/3)$	$-7/3 \approx -2.33$	Exact Minimizer
Optimal	$(1/3, 4/3)$	$-7/3 \approx -2.33$	Target

Key Observation: With exact line search, the DFP method reaches the minimizer in two iterations for this two-dimensional quadratic problem.

Numerical Example: Summary

After the first update, the inverse Hessian approximation changes from

$$H^{(0)} = I \quad \longrightarrow \quad H^{(1)} \approx \begin{bmatrix} 0.6716 & -0.3377 \\ -0.3377 & 0.6705 \end{bmatrix}.$$

Why this matters:

- ▶ The nonzero off-diagonal terms indicate that the method has learned the interaction between x and y .
- ▶ As a result, the second search direction points much more directly toward the minimizer.
- ▶ With exact line search, the DFP method reaches the exact solution in the second iteration.

Progress Check

$$f(x^{(0)}) = 0 \quad \longrightarrow \quad f(x^{(1)}) \approx -2.2237 \quad \longrightarrow \quad f(x^{(2)}) = -\frac{7}{3} \approx -2.3333$$

The BFGS Update

Among quasi-Newton methods, the **BFGS method** (Broyden–Fletcher–Goldfarb–Shanno method) is the most widely used.

Like the DFP method, BFGS updates an approximation of the inverse Hessian matrix. Let $H^{(k)}$ denote the current approximation of $(\nabla^2 f(x^{(k)}))^{-1}$. Then the search direction is

$$p^{(k)} = -H^{(k)} \nabla f(x^{(k)}).$$

After defining

$$s^{(k)} = x^{(k+1)} - x^{(k)}, \quad y^{(k)} = \nabla f(x^{(k+1)}) - \nabla f(x^{(k)}),$$

the BFGS update is

$$H^{(k+1)} = \left(I - \rho_k s^{(k)} (y^{(k)})^T \right) H^{(k)} \left(I - \rho_k y^{(k)} (s^{(k)})^T \right) + \rho_k s^{(k)} (s^{(k)})^T,$$

where

$$\rho_k = \frac{1}{(y^{(k)})^T s^{(k)}}.$$

Properties of the BFGS Update

The BFGS update is preferred in practice because of its strong numerical performance.

- ▶ It satisfies the secant condition

$$H^{(k+1)}y^{(k)} = s^{(k)}.$$

- ▶ If $H^{(k)}$ is symmetric and positive definite, and if

$$(y^{(k)})^T s^{(k)} > 0,$$

then $H^{(k+1)}$ is also symmetric and positive definite.

- ▶ Therefore, the search direction

$$p^{(k)} = -H^{(k)}\nabla f(x^{(k)})$$

is a descent direction.

For these reasons, BFGS is often more reliable than DFP in practical optimization problems.

The BFGS Algorithm

Step 1. Choose an initial point $x^{(0)}$, an initial symmetric positive definite matrix $H^{(0)}$, and set $k = 0$.

Step 2. Compute the gradient $\nabla f(x^{(k)})$.

Step 3. If $\nabla f(x^{(k)}) = 0$, or if a selected stopping criterion is satisfied, stop.

Step 4. Compute the search direction $p^{(k)} = -H^{(k)}\nabla f(x^{(k)})$.

Step 5. Choose a step size α_k using a line search procedure.

Step 6. Update $x^{(k+1)} = x^{(k)} + \alpha_k p^{(k)}$.

Step 7. Define

$$s^{(k)} = x^{(k+1)} - x^{(k)}, \quad y^{(k)} = \nabla f(x^{(k+1)}) - \nabla f(x^{(k)}).$$

Step 8. Update $H^{(k)}$ by the BFGS formula, set $k \leftarrow k + 1$, and return to Step 2.

Numerical Example: BFGS - Step 1

We minimize the same function $f(x, y) = x^2 + y^2 + xy - 2x - 3y$ starting from $x^{(0)} = [0, 0]^T$ with $H^{(0)} = I$.

Initial Step: Since $H^{(0)} = I$, the first step of BFGS is identical to DFP (and Steepest Descent):

- ▶ $p^{(0)} = [2, 3]^T$
- ▶ $\alpha_0 = 13/38$ (from exact line search)
- ▶ $x^{(1)} \approx [0.6842, 1.0263]^T$

Calculated Differences:

$$s^{(0)} = \begin{bmatrix} 13/19 \\ 39/38 \end{bmatrix}, \quad y^{(0)} = \begin{bmatrix} 91/38 \\ 52/19 \end{bmatrix}, \quad (y^{(0)})^T s^{(0)} = \frac{169}{38} \approx 4.45$$

Numerical Example: BFGS - Step 2 (The Scale)

Unlike DFP, BFGS uses a scaling factor ρ_k to ensure stability:

$$\rho_0 = \frac{1}{(y^{(0)})^T s^{(0)}} = \frac{38}{169} \approx 0.2249.$$

The update formula for $H^{(1)}$ is:

$$H^{(1)} = \underbrace{(I - \rho_0 s^{(0)}(y^{(0)})^T)}_V H^{(0)} \underbrace{(I - \rho_0 y^{(0)}(s^{(0)})^T)}_{V^T} + \underbrace{\rho_0 s^{(0)}(s^{(0)})^T}_W$$

The Structure: Notice the "sandwich" form $(VIV^T + W)$. This ensures $H^{(1)}$ stays ****positive definite**** even with rounding errors.

Numerical Example: BFGS - Step 3 (The Second Iteration)

Now we compute the second BFGS direction using

$$H^{(1)} \approx \begin{bmatrix} 0.6814 & -0.3463 \\ -0.3463 & 0.6780 \end{bmatrix}$$

and

$$\nabla f(x^{(1)}) = \begin{bmatrix} 15/38 \\ -5/19 \end{bmatrix} \approx \begin{bmatrix} 0.39 \\ -0.26 \end{bmatrix}.$$

The new search direction is

$$p^{(1)} = -H^{(1)}\nabla f(x^{(1)}) \approx \begin{bmatrix} -0.3601 \\ 0.3151 \end{bmatrix}.$$

Using exact line search, $\alpha_1 \approx 0.9744$.

Hence

$$x^{(2)} = x^{(1)} + \alpha_1 p^{(1)} = \begin{bmatrix} 0.68 \\ 1.03 \end{bmatrix} + 0.9744 \begin{bmatrix} -0.3601 \\ 0.3151 \end{bmatrix} = \begin{bmatrix} 0.33 \\ 1.33 \end{bmatrix}.$$

Thus, the BFGS method reaches the exact minimizer at the second iteration.

Numerical Example: BFGS Result

After substituting the values and simplifying:

$$H_{BFGS}^{(1)} \approx \begin{bmatrix} 0.6814 & -0.3463 \\ -0.3463 & 0.6780 \end{bmatrix}$$

Comparison with DFP:

- ▶ **DFP Result:** $H^{(1)} \approx \begin{bmatrix} 0.6716 & -0.3377 \\ -0.3377 & 0.6705 \end{bmatrix}$.
- ▶ **BFGS Result:** $H^{(1)} \approx \begin{bmatrix} 0.6814 & -0.3463 \\ -0.3463 & 0.6780 \end{bmatrix}$

Key Discussion: While both approximations are symmetric and positive definite, BFGS is generally more robust to numerical inaccuracies. In this step, the BFGS and DFP approximations are quite close, and both capture the curvature information effectively.

Why BFGS is Popular

In practice, BFGS is often preferred because it combines efficiency and robustness.

- ▶ It avoids computing the Hessian matrix explicitly.
- ▶ It updates the inverse Hessian approximation using only gradient information.
- ▶ It usually converges much faster than the steepest descent method.
- ▶ It is often more stable and effective than the DFP method.

For large-scale optimization problems, quasi-Newton methods such as BFGS are among the most widely used methods.

Homework I

Consider the quadratic function

$$f(x, y) = 2x^2 + y^2 - 2xy + 2x - 4y.$$

Starting from $x^{(0)} = [0 \ 0]^T$, solve the following.

1. Apply **Newton's method** using the formula

$$x^{(1)} = x^{(0)} - (\nabla^2 f(x^{(0)}))^{-1} \nabla f(x^{(0)}).$$

In this part, explicitly compute the inverse Hessian matrix and then find the next point.

2. Apply **Newton's method** again, but this time compute the Newton direction $p^{(0)}$ by solving the linear system

$$\nabla^2 f(x^{(0)}) p^{(0)} = -\nabla f(x^{(0)}),$$

and then update the next point.

3. Show that the two approaches in parts (1) and (2) give the same iterate.
4. Verify that the obtained point is the exact minimizer of f .

Homework II

Consider the function

$$f(x, y) = e^x + \frac{1}{2}y^2 - 2x - y$$

with initial point

$$x^{(0)} = \begin{bmatrix} -1 \\ 1 \end{bmatrix}.$$

For the quasi-Newton methods, take $H^{(0)} = I$ and use the prescribed step size $\alpha_0 = \frac{1}{2}$.

Solve the following:

1. Apply one iteration of **Newton's method** and compute $x^{(1)}$.
2. Apply one iteration of the **damped Newton method** using backtracking line search, and determine the accepted step size α_0 ($\rho = \frac{1}{3}$, $c = 10^{-4}$).
3. Apply one iteration of the **DFP method**. Compute $p^{(0)}$, $x^{(1)}$, $s^{(0)}$, $y^{(0)}$, and $H^{(1)}$.
4. Apply one iteration of the **BFGS method**. Compute $p^{(0)}$, $x^{(1)}$, $s^{(0)}$, $y^{(0)}$, and $H^{(1)}$.

Appendix: What is a Rank-1 Update?

In linear algebra, a **Rank-1 matrix** is formed by the outer product of two vectors u and v :

$$A = uv^T$$

Why "Rank-1"?

- ▶ Every column of A is a multiple of u .
- ▶ It contains information along only **one specific direction**.

In Optimization: Instead of recomputing the $n \times n$ Hessian from scratch ($O(n^3)$), we "update" the previous matrix by adding a Rank-1 term ($O(n^2)$):

$$H^{(k+1)} = H^{(k)} + \alpha(uu^T)$$

This adds new curvature information gained from the latest step.

Appendix: Rank-1 Update Numerical Example

Let $u = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$ and $v = \begin{bmatrix} 3 \\ 4 \end{bmatrix}$. Then:

$$uv^T = \begin{bmatrix} 1 \cdot 3 & 1 \cdot 4 \\ 2 \cdot 3 & 2 \cdot 4 \end{bmatrix} = \begin{bmatrix} 3 & 4 \\ 6 & 8 \end{bmatrix}$$

Insight: Notice that the second row is exactly $2 \times$ the first row. The matrix has only **one independent direction** of information. Adding this to $H^{(k)}$ updates the Hessian approximation along that specific direction.

Appendix: DFP as a Rank-2 Update

The DFP formula is a **Rank-2 update** because it adds two distinct Rank-1 matrices to $H^{(k)}$:

$$H^{(k+1)} = H^{(k)} + \underbrace{\frac{s^{(k)}(s^{(k)})^T}{(s^{(k)})^T y^{(k)}}}_{\text{Term 1 (Rank-1)}} - \underbrace{\frac{(H^{(k)} y^{(k)})(H^{(k)} y^{(k)})^T}{(y^{(k)})^T H^{(k)} y^{(k)}}}_{\text{Term 2 (Rank-1)}}$$

Why two terms?

- **Correction: Term 1** ensures the **Secant Condition** is met:
 $H^{(k+1)} y^{(k)} = s^{(k)}.$
- **Damping: Term 2** subtracts the portion of $H^{(k)}$ that is inconsistent with the new gradient information. This ensures that $H^{(k+1)}$ remains **positive definite**, guaranteeing that each search direction is a descent direction.

The Result: By summing these two "rank-1" pieces, we get a **Rank-2** change that preserves both *symmetry* and *positive definiteness*.

Appendix: Newton vs. Quasi-Newton (Cost Analysis)

Why do we prefer this "rank-based" update over the original Newton's method?

Feature	Newton's Method	Quasi-Newton (DFP/BFGS)
Information	2nd Order (Exact)	1st Order (Approximated)
Hessian Cost	$O(n^2)$ evaluations	Zero (only gradients)
Linear System	$O(n^3)$ (Inversion)	$O(n^2)$ (Direct Multiply)
Storage	$n \times n$ matrix	$n \times n$ matrix (or less for L-BFGS)

Conclusion: Even if Newton's method converges in fewer steps, each step in Quasi-Newton is so much "cheaper" that the total clock time is often significantly lower for large n .

Appendix: Visualizing the Secant Approximation

The Secant Condition $B^{(k+1)}s^{(k)} = y^{(k)}$ is a multi-dimensional generalization of the 1D slope formula:

$$f''(x) \approx \frac{f'(x_{k+1}) - f'(x_k)}{x_{k+1} - x_k} = \frac{y_k}{s_k}$$

Geometric Intuition: Imagine the objective function $f(x)$ as a bowl.

- ▶ **Newton's Method** measures the exact curvature (Hessian) at a **single point**, requiring expensive second-order derivatives.
- ▶ **Quasi-Newton Methods** "feel" the slope at point $x^{(k)}$ and then at $x^{(k+1)}$. By observing how the gradient (∇f) changes over the interval $s^{(k)}$, they **infer** the curvature.

Why call it a "Learning Process"? Instead of being given the geometry (Hessian) upfront, the algorithm constructs a local model of the function's shape through exploration. This adaptive nature is why they are also known as **variable metric methods**.

Appendix: Why Newton's Method is $O(n^3)$

Solving the Newton system $\nabla^2 f(x)p = -\nabla f(x)$ requires **matrix factorization** (e.g., LU or Cholesky decomposition).

The Cubic Bottleneck:

- ▶ To eliminate one variable, we must perform operations across n rows and n columns.
- ▶ Since there are n variables to eliminate, the total operations scale as $n \times n \times n = n^3$.

Comparison for $n = 1,000$ variables:

- ▶ **Newton** (n^3): $\approx 1,000,000,000$ (1 Billion) operations.
- ▶ **Quasi-Newton** (n^2): $\approx 1,000,000$ (1 Million) operations.

Insight: Quasi-Newton is **1,000 times faster** per iteration for large-scale problems.

Appendix: Storage and Memory Efficiency

In high-dimensional optimization (e.g., Deep Learning), storing the exact Hessian matrix is often impossible.

Memory Requirements ($n \times n$):

- ▶ For $n = 10^5$ variables, a full Hessian takes ≈ 40 GB of RAM.
- ▶ Newton's method requires storing and manipulating this massive matrix every step.

The Quasi-Newton Advantage:

- ▶ We update the approximation using only vectors $s^{(k)}$ and $y^{(k)}$ (size n).
- ▶ **L-BFGS (Limited-memory BFGS)**: Even avoids storing the $n \times n$ matrix entirely, using only the last m pairs of (s, y) to compute the direction.

This makes Quasi-Newton methods the only viable "Newton-like" option for **Big Data** problems.

Appendix: Optimization Method Trade-offs

Choosing the right method is a balance between **Convergence Rate** and **Computational Cost**.

Method	Convergence	Cost/Iter.	Memory
Steepest Descent	Linear	$O(n)$	$O(n)$
Newton	Quadratic	$O(n^3)$	$O(n^2)$
Quasi-Newton	Superlinear	$O(n^2)$	$O(n^2)$
L-BFGS	Superlinear	$O(mn)$	$O(mn)$

Key Takeaway: Newton's method is the "fastest" in terms of steps, but Quasi-Newton (BFGS) is the "fastest" in terms of **total CPU time** for most practical engineering applications.